Towards a Universal Feature Extractor for Atari Games: Reconstruction, Finite Scalar Quantisation and Joint-Embedding Prediction

Sajid Ali Sadruddin

TU Dresden, Germany
Research Project, supervised by Oleh Shkalikov

Abstract. Reinforcement learning (RL) agents for Atari usually learn a separate convolutional feature extractor for each game, although all games provide the same type of observation: four stacked 84×84 grey-scale frames. In this project, I investigated whether a single encoder, pre-trained once on expert gameplay from eleven games, can be reused as a universal feature extractor for Proximal Policy Optimisation (PPO). I compared a Gaussian VAE and autoencoders with vector-quantised and Finite Scalar Quantisation (FSQ) bottlenecks against an action- and game-conditioned Joint-Embedding Predictive Architecture (V-JEPA) world model. Each encoder was either kept frozen or fine-tuned together with the policy for 10 M environment steps on the eleven pre-training games and four unseen games. The fine-tuned FSQ-VAE achieved a higher median human-normalised score than a NatureCNN trained from scratch (1.07 vs. 0.74), outperformed it on four of eleven games and nearly matched it on two more, and on four games reached 80% of its final level 1.4–2.6 M steps earlier. When frozen, the same encoder reached less than a third of that median. On the unseen games, the NatureCNN stayed ahead on three of four. The V-JEPA learned a non-collapsed latent space and meaningful dynamics on the seen games, but PPO was barely able to use its features (median 0.05). The diagnostics show that the model largely ignored the action, produced nearly identical latents for consecutive frames and specialised to the pre-training games. These results suggest that a shared pre-trained encoder is useful as an initialisation, but not yet as a frozen replacement for the NatureCNN.

1 Introduction

Atari 2600 games, played through the Arcade Learning Environment [1], are a standard benchmark for learning control from pixels. Since DQN [2], the standard agent stacks the last four pre-processed frames and passes them through a small convolutional network, the NatureCNN, followed by a policy and a value head. The network is trained from the reward alone, separately for every game. This is inefficient, since the games share the same input format and a similar visual vocabulary of sprites, balls, walls and scrolling backgrounds (Figure 2). An

extractor trained once and then reused could save interactions and computation, transfer to new games, and provide one latent space for all games.

This project, proposed by Oleh Shkalikov, asks whether such a *universal feature extractor* can be trained for Atari. I studied two aspects. The first is the pre-training objective: I used quantised autoencoders [3,4], which learn by reconstructing the frames, and a joint-embedding predictive architecture (JEPA) [5,6,7], which learns by predicting the next state in latent space. The second is how the extractor is used: kept frozen, or fine-tuned with the policy. The project addresses four research questions: which objective gives the most useful features for PPO (**RQ1**), whether freezing or fine-tuning works better (**RQ2**), whether the features generalise to games that were not seen during pre-training (**RQ3**), and what the approach costs and how interpretable it is (**RQ4**).

2 Related Work

PPO [8] with the RL-Zoo hyper-parameters [9] is the standard on-policy baseline for Atari. Many multi-game agents share the entire policy across games, whereas I only share the perception part, so that the effect of the representation can be measured in isolation. Discrete latent spaces have been used successfully on Atari (DreamerV2 [10], IRIS [11]), and self-predictive methods such as ATC [12] and SPR [13] avoid pixel reconstruction altogether. ATC also showed that a reward-free encoder can be kept frozen for some Atari games. In robotics, pre-trained encoders have been reported both as highly effective [14] and as no better than an encoder trained from scratch [15], a tension my results also show. For the JEPA, I adopted the learnable task embeddings of the TD-MPC2 world model [16].

3 Prerequisites

Problem Formulation. In every game g, the agent sees a frame stack $x_t \in [0,1]^{4\times84\times84}$ and can choose from its own set of 4 to 18 actions. A feature extractor f_θ maps x_t to a feature vector, on which PPO trains an actor and a critic. The goal is one set of weights $\theta^\star$, learned without rewards on eleven games, that works for PPO on any game. I compare a NatureCNN trained from scratch (the baseline) with a pre-trained encoder that is either kept *frozen* or *fine-tuned* with the policy.

Dataset. A random policy rarely gets past the first screen of a game, so I collected the data with pre-trained agents from the HuggingFace Hub (Figure 1a). Their actions were sampled from their policies, so the data also contains some sub-optimal behaviour. I built two datasets. The *frame dataset* for the autoencoders contains 50 k frame stacks per game, which gives 550 k stacks from the eleven pre-training games and 200 k from the four held-out games. The *transition dataset* for the world model contains tuples (s_t, a_t, s_{t+1}, d_t): 250 k per pre-training game (2.75 M in total), a validation split of 10 k per game and 50 k per held-out game. All data was collected with seed 42. At the end of an episode, the actual last

frame is stored instead of the reset frame. Consecutive stacks share three of their four frames, so the next state is already 75% identical to the current one, and episode-ending transitions are masked out (Figure 1b).

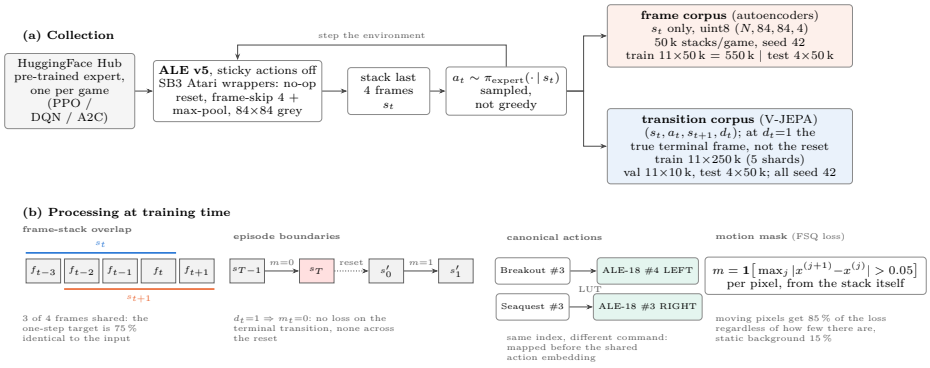


Fig. 1: (a) Data collection with pre-trained expert agents and the two resulting corpora. (b) Processing at training time: frame-stack overlap, masking of episode-ending transitions, mapping of game-specific action indices to the canonical ALE-18 set, and the motion mask of the FSQ-VAE loss.

4 The Games

Figure 2 shows all 15 games as the encoder sees them. Eleven are used for pre-training and four are held out to test generalisation. Every game produces a tensor of the same shape, but the content varies considerably (Table 1), and a shared encoder has to cope with all of these differences at once.

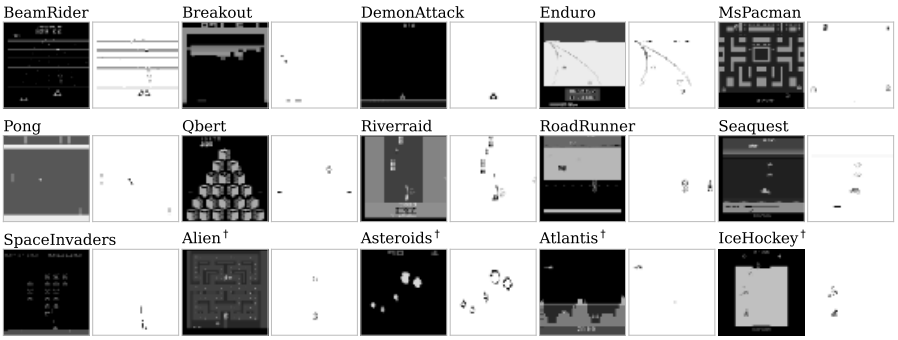


Fig. 2: All 15 games as the encoder sees them: the newest frame t of an expert frame stack (left of each pair) and $|x_{t+1} - x_t|$, the change to the next frame (right, inverted, per-image contrast). † held-out games. Frames from the data specimen sheet of the transition corpus.

Table 1: How the games differ. $|\mathcal{A}|$: minimal action set; pixel mean (0–255) and black % of the frames; moving %: pixels that change within a stack ($\delta = 0.05$); steps per terminal (the Atari wrappers also end an episode on a life loss); top action: share of the expert's most frequent action. Computed on 50 k transitions per game of the transition corpus; † held-out.

Game	Type	$\lvert\mathcal{A}\rvert$	Expert	Pix. mean	Black %	Mov. %	Steps/ term.	Top act. %
BeamRider	pseudo-3D shooter	9	PPO	13	79.0	33.5	1 163	29
Breakout	paddle, ball, bricks	4	PPO	33	71.3	1.4	476	28
DemonAttack	fixed shooter	6	DQN	5	87.2	3.7	575	24
Enduro	pseudo-3D racing	9	PPO	53	35.4	6.9	12 500	31
MsPacman	maze	9	PPO	56	19.7	3.5	301	26
Pong	paddle vs. paddle	6	PPO	105	1.0	1.7	1 724	22
Qbert	isometric platform	6	PPO	35	63.1	2.2	472	45
Riverraid	vertical scroller	18	PPO	87	8.9	10.3	265	13
RoadRunner	horizontal scroller	18	A2C	83	45.2	7.4	303	67
Seaquest	underwater shooter	18	PPO	48	16.1	3.8	562	8
SpaceInvaders	fixed shooter	6	PPO	9	85.5	2.8	510	28
Alien†	maze	18	DQN	49	14.9	3.2	213	38
Asteroids†	free-roaming shooter	14	DQN	6	95.0	6.0	145	99
Atlantis†	fixed defence	4	DQN	36	60.2	6.1	118	66
IceHockey†	team sport	18	DQN	78	55.6	1.1	3 333	30

Appearance. The mean brightness ranges from 5 (DemonAttack) to 105 (Pong), and the share of black pixels from 1% (Pong) to 95% (Asteroids). Shooters on a black background (BeamRider, DemonAttack, SpaceInvaders, Asteroids) are 79–95% black, whereas Pong, Riverraid, Seaquest and the maze games fill most of the screen (1–20% black).

Motion. In most games, fewer than 4% of the pixels change within a frame stack (Pong 1.7%, Breakout 1.4%, IceHockey 1.1%). Only BeamRider (34%) and Riverraid (10%) move large parts of the screen; the other scrolling and pseudo-3D games (RoadRunner 7.4%, Enduro 6.9%) change about as much as Asteroids and Atlantis (6%). The objects relevant for playing are thus usually a few small sprites on a static background, which a loss that weights all pixels equally can almost ignore.

Actions and episode length. The number of actions ranges from 4 to 18, and the same action index means a different joystick command in different games, so a shared action embedding requires a common action set (Section 5). Episodes last from about 120 steps (Atlantis) to 12 500 steps (Enduro). The expert data is also unevenly varied: the most frequent expert action covers 8% of the steps in Seaquest, but 67% in RoadRunner and 99% in Asteroids, so for such games the data rarely shows the effect of a different action.

5 Models

All encoders receive the same input $x \in [0, 1]^{4\times84\times84}$. After pre-training, only the encoder is kept, and its output is passed to the actor and critic of PPO. The baseline is a NatureCNN with a 512-dimensional feature, trained from scratch.

5.1 Autoencoders

I first trained a Gaussian VAE [17] with the NatureCNN layers as encoder and a 256-dimensional latent vector (Table 3). Its encoder reduces the frame to a 7×7 feature map before the bottleneck, so each cell is responsible for a 12×12 pixel region, much larger than a ball or a bullet. I therefore also trained spatial, discrete bottlenecks.

Residual VQ-VAE. This model keeps a finer 21×21 latent grid, using two stride-2 convolutions and two residual blocks. Each 64-dimensional latent cell is replaced by the closest of $K = 512$ codebook vectors, and the gradient is passed through with the straight-through trick. The codebook is updated with exponential moving averages ($\gamma = 0.99$), the commitment weight is $\beta = 0.25$, and codes that are no longer used are re-initialised. Inspecting the reconstructions showed that small moving objects were often missing. To address this, I trained a second version in which the error on moving pixels, $m = \mathbf{1}[\max_j |x^{(j+1)} - x^{(j)}| > 0.05]$, is weighted ten times higher.

FSQ-VAE. Finite Scalar Quantisation [4] removes the codebook entirely. A three-layer convolutional encoder outputs four channels, and each channel is rounded to a fixed number of levels, $L = [8, 5, 5, 3]$, giving 600 possible values per latent cell. There is no codebook, no commitment loss and no unused codes. My first version used transposed convolutions in the decoder, which caused checkerboard artefacts on the flat Atari backgrounds, so I replaced them with upsampling followed by standard convolutions. Instead of a fixed factor of ten, the final loss normalises moving and static pixels separately:

$$\mathcal{L}_{\text{bal}} = w \, \frac{\sum m \odot (\hat{x} - x)^2}{\sum m} + (1 - w) \, \frac{\sum (1 - m) \odot (\hat{x} - x)^2}{\sum (1 - m)}, \qquad w = 0.85. \tag{1}$$

Moving pixels thus always account for 85% of the loss, however few of them there are. For PPO, the feature is the flattened $4 \times 21 \times 21$ quantised grid (1 764 values).

5.2 Action- and Game-Conditioned V-JEPA

A JEPA does not reconstruct pixels. Instead, it learns to predict the *representation* of the next observation [5]. If the predictor is also given the action, the model becomes a latent world model, similar to SPR [13]. Figure 3 shows the final model. A 14×14 patch embedding turns a frame stack into 36 tokens of size 256. The context encoder E_θ, a six-layer transformer, encodes s_t, and the target encoder $\bar{E}$, an exponential moving average (EMA) of E_θ with $\tau = 0.996$ that receives no gradients [6], encodes s_{t+1}. A three-layer predictor P_ψ gets the context tokens together with an action token and a game token, and has to predict the target tokens:

$$\mathcal{L}_{\text{JEPA}} = \frac{\sum_i m_i \left\| P_\psi\big(E_\theta(s_t^i, e^i), a_t^i, e^i\big) - \text{sg}\left(\bar{E}(s_{t+1}^i, e^i)\right) \right\|_2^2}{\sum_i m_i}, \tag{2}$$

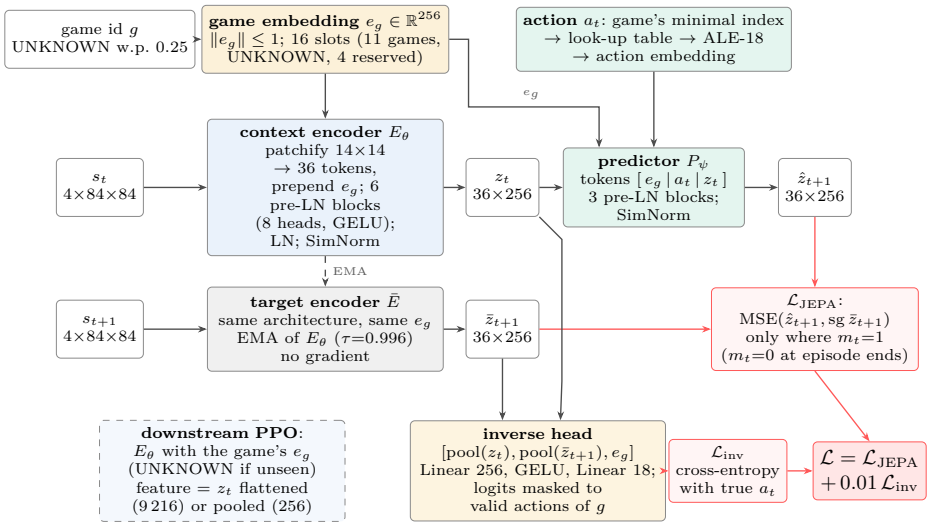


Fig. 3: Environment- and action-conditioned V-JEPA (v2), as implemented. The target encoder is an EMA copy of the context encoder; only the context encoder is kept for PPO.

where $m_i = 0$ for transitions that end an episode.

My first version had a good loss curve, but a review showed that action indices meant different commands in different games, the large dataset was loaded with the wrong axis order, and the validation set contained only one game. I therefore rebuilt the model (v2) following TD-MPC2 [16]. First, all actions are mapped to the common set of 18 ALE actions. Second, each game receives a learnable embedding e_g with $\|e_g\| \le 1$, which is added as a token to both encoders and the predictor. With probability 0.25 it is replaced by a shared UNKNOWN embedding, so that the model also learns a general "Atari" prior for unseen games. Third, all latents are normalised with SimNorm, which splits them into 32 groups of $V = 8$ values and applies a softmax to each group. This keeps the latents bounded and makes collapse harder. Fourth, an inverse-dynamics head [18] predicts the action a_t from z_t, $\bar{z}_{t+1}$ and e_g, only among the actions that are valid in the game. The total loss is $\mathcal{L} = \mathcal{L}_{\text{JEPA}} + \lambda \mathcal{L}_{\text{inv}}$ with $\lambda = 0.01$.

Training Step. The target branch is computed without gradients, both losses are averaged over the same unmasked samples, the gradients are clipped to a norm of 1.0, AdamW updates the trainable parameters, and only then is the target encoder moved slightly towards the context encoder (Table 2). The target is thus a slowly moving goal that the context encoder cannot change directly, which prevents collapse to a constant output. The inverse loss trains the encoder but *not* the predictor, because its second input is the target latent rather than the prediction.

Table 2: Gradient routing in the V-JEPA (12.48 M parameters, 7.53 M trainable).

Parameter group	Params	$\nabla\mathcal{L}_{\text{JEPA}}$	$\nabla\mathcal{L}_{\text{inv}}$	Update
Context encoder (patchify, 6 blocks, LN)	4.95 M	yes	yes	AdamW
Predictor (action emb., 3 blocks, LN)	2.37 M	yes	no	AdamW
Inverse head ($768 \to 256 \to 18$)	0.20 M	no	yes	AdamW
Game embeddings (16×256)	0.004 M	yes	yes	AdamW
Target encoder (copy of context enc.)	4.95 M	no	no	EMA only

Using the Encoder in PPO. The context encoder receives the game's own embedding (or UNKNOWN for unseen games) and outputs 36×256 tokens, which are flattened into a 9 216-dimensional feature in the main runs and averaged into a 256-dimensional feature in a second set.

6 Methodology

Table 3 lists the training setup of each encoder next to the setting of the paper it is based on. The architectures follow the references, and most differences are due to scale: the JEPA papers train large vision transformers on very large datasets, whereas my model has 12.5 M parameters and was trained on a single GPU, so I used a smaller batch, a lower learning rate, less weight decay and a fixed EMA momentum. For the V-JEPA, I selected the checkpoint with the lowest validation loss after the early, near-collapsed phase (epoch 90 of 92; Section 7.1), and a small sweep over $\lambda \in \{0.003, 0.01, 0.03, 0.1\}$ confirmed $\lambda = 0.01$.

All PPO agents use the standard RL-Zoo Atari settings (last column of Table 3, with $\gamma = 0.99$, $\lambda_{\text{GAE}} = 0.95$, clip range 0.1 and entropy coefficient 0.01) and the same 512-unit actor and critic heads. All agents were trained using Stable-Baselines3. Performance is the mean training return over the last 10% of training (steps 9–10 M). As score ranges differ widely between games, I convert it to the human-normalised score HNS$=$ (score$-$random)/(human$-$random) with the reference scores of Badia et al. [19] and mainly report the median over games. All numbers except the Std-VAE and Res-VQ scores and the NatureCNN and V-JEPA scores on the held-out games, which I only recorded in rounded form, are computed from the complete logs in Weights & Biases. Each configuration was run with only one seed, so small differences should be interpreted with caution [20].

Table 3: Training configurations of the four encoders used with PPO ("this work") and the setting of the reference paper each follows. "–": not specified, or not logged.

	Standard VAE	Residual VQ-VAE	FSQ-VAE	V-JEPA v2	NatureCNN
This work					
Objective	MSE + KL div.	MSE[a] + VQ + commit.	$\mathcal{L}_{\text{bal}}$, $w = 0.85$	$\mathcal{L}_{\text{JEPA}} + 0.01\,\mathcal{L}_{\text{inv}}$	PPO only (no pre-training)
Data	550 k stacks	550 k stacks	550 k stacks	2.75 M transitions	online, 10^7 steps/game
Length	58 ep. (250 k steps)	29 ep. (250 k steps)	125 ep. (537 k steps)	92 ep. (494 k steps)	9.8 k rollouts
Batch	128	64	128	512	8×128; minibatch 256
Optimiser, LR	Adam, 10^{-3}	$3{\cdot}10^{-4}$	AdamW, $3{\cdot}10^{-4}$	AdamW, $3{\cdot}10^{-4}$	Adam, $2.5{\cdot}10^{-4} \to 0$
Weight decay	–	–	10^{-4}	10^{-5} (not LN/emb.)	0
Schedule	–	–	100 ep. const. + cosine	warm-up + cosine	linear to 0
Bottleneck / target	μ, σ d=256	K=512, d=64, β=0.25, EMA 0.99	FSQ L=[8, 5, 5, 3] (600)	EMA τ=0.996 (fixed); SimNorm V=8; patch 14	4 epochs, VF coef. 0.5
Parameters	2.56 M	0.70 M	0.84 M	12.5 M	1.68 M
Hardware, time	H100, 0.8 h	H100, 0.9 h	H100, 1.9 h[b]	H100, 15.6 h	H100, 3.0 h/game
Reference setting					
Source	VAE [17]	VQ-VAE [3]	FSQ [4]	I-JEPA [6] / V-JEPA [7] / TD-MPC2 [16]	PPO [8], RL-Zoo [9]
Length	–	250 k steps	250 k steps	300 ep. / 90 k it. / –	10^7 steps (40 M frames)
Batch	100	128	–	2 048 / 3 072 / 256	8×128; minibatch 32×8
Optimiser, LR	Adagrad, 0.01–0.1	Adam, $2{\cdot}10^{-4}$	–	AdamW, 10^{-3} / $6.25{\cdot}10^{-4}$ / Adam, $3{\cdot}10^{-4}$	Adam, $2.5{\cdot}10^{-4} \cdot \alpha$
Weight decay	–	–	–	$0.04 \to 0.4$ (I-, V-JEPA)	–
Bottleneck / target	μ, σ $d \leq 200$	K=512, β=0.25, EMA 0.99	[8, 6, 5] (2^8), [8, 5, 5, 5] (2^{10})	EMA $0.996 \to 1$ / $0.998 \to 1$; SimNorm V=8; patch 14 / 16	3 epochs, VF coef. 1 (Zoo: 4, 0.5)

[a] plain MSE, or with moving pixels weighted 10× (sprite-weighted variant). [b] estimated from 0.9 h per 250 k steps.

7 Results

7.1 Pre-Training

The pre-training curves revealed two unexpected effects. First, the held-out error was lowest early in training. The FSQ-VAE had its lowest held-out error after only three epochs, and after 125 epochs this error was 21% higher. For the V-JEPA, the final held-out loss was three times higher than on the seen games, and the held-out inverse accuracy fell below chance. Both models thus specialised to the pre-training games. Second, a rising loss was not a bad sign for the V-JEPA. Its validation loss was lowest in the first epoch and increased afterwards, while

the effective rank of its latents grew from 1.4 after the first epoch, below the 2.5 of an untrained model, to 57 (out of 256), and the inverse accuracy from 23% to 58% (chance: 13%). The low initial loss came from an almost collapsed latent space, where prediction is trivial. These per-epoch values were measured on the validation split; the diagnostics in Section 7.3 use 64 five-step sequences, so their values differ slightly.

I evaluated reconstruction on 256 stacks per game. On the seen games, the FSQ-VAE achieved the lowest error ($0.37 \cdot 10^{-3}$) and a three times lower error on moving pixels than the plain VQ-VAE, which shows that the balanced loss and the new decoder improved the reconstruction of small sprites. Weighting the sprites in the VQ-VAE reduced the moving-pixel error by 27%, but increased the overall error by 19%. On the held-out games, all errors were four to eleven times higher and the FSQ-VAE performed worst, so none of the autoencoders generalised to new games at the pixel level.

7.2 Downstream Reinforcement Learning

FSQ-VAE, Fine-Tuned. Fine-tuning the pre-trained FSQ encoder produced the best pre-trained agent and the highest median HNS of all agents (1.07 vs. 0.74 for the NatureCNN; Table 4 and Figure 4). It outperformed the NatureCNN on Breakout, Enduro, Qbert and Riverraid, and nearly matched it on Pong (20.7 vs. 21.0) and Seaquest (1796 vs. 1801). The NatureCNN remained ahead on the other five games: by 8–21% on MsPacman, BeamRider and SpaceInvaders, and by about a factor of two on DemonAttack and RoadRunner, which explains its higher mean HNS (2.51 vs. 2.02). Pre-training also sped up learning (Table 5): on Enduro, MsPacman, Qbert and Riverraid, the fine-tuned agent reached 80% of the NatureCNN's final level 1.4–2.6 M steps (19–39%) earlier, and it reached the 50% level faster on five games.

FSQ-VAE, Frozen. With a frozen FSQ encoder, the agent still made progress in every game, but considerably less (median HNS 0.31). It came close to the NatureCNN only on Pong and MsPacman. Performance was weakest in games where the relevant object is small and fast (DemonAttack, Breakout, BeamRider), where the agent reached only 6–11% of the NatureCNN's HNS, compared with 35–57% on the remaining games. Even with the motion-aware loss, reconstruction still devotes most of its capacity to large areas.

Standard VAE and Residual VQ-VAE. The Standard VAE scored lowest among the autoencoders on every game, both frozen and fine-tuned (median HNS 0.06 and 0.36). The Residual VQ-VAE scored below the FSQ-VAE on every game except MsPacman (fine-tuned), but fine-tuned it still reached a higher median HNS than the NatureCNN (0.85 vs. 0.74). The gap to the FSQ-VAE was largest on Breakout, DemonAttack and BeamRider, where the fine-tuned Residual VQ-VAE reached only 28–52% of the FSQ-VAE's return. These games have small, fast-moving objects, which is consistent with its weaker reconstruction of moving pixels (Section 7.1).

Table 4: Mean episode return over the last 10% of 10 M training steps (single seed); best per game in bold. † held-out games, not used for pre-training. Std-VAE and Res-VQ scores, and NatureCNN and V-JEPA scores on the held-out games, are rounded as recorded. V-JEPA with the flattened feature; the pooled variant is in Table 6.

	NatureCNN	Std-VAE		Res-VQ		FSQ-VAE		V-JEPA	
Game	scratch	frozen	joint	frozen	joint	frozen	joint	frozen	joint
BeamRider	**3610**	250	1100	310	1600	709	3077	570	615
Breakout	306	5.5	45.0	9.0	90.0	26.9	**322**	9.6	7.8
DemonAttack	**12.9k**	180	1100	350	2200	935	6914	331	205
Enduro	638	50	450	150	800	267	**918**	39.0	43.5
MsPacman	1991	850	1200	1500	**2100**	1952	1841	693	617
Pong	**21.0**	-5.0	12.0	12.0	19.5	18.7	20.7	-20.2	-15.0
Qbert	13.9k	1200	5000	3200	11.5k	4918	**15.9k**	613	323
Riverraid	8152	1500	3800	2800	5500	5231	**8487**	1989	1863
RoadRunner	**43.0k**	6000	11.5k	9500	16.0k	16.3k	20.7k	5988	1033
Seaquest	**1801**	350	850	650	1650	881	1796	495	555
SpaceInvaders	**1107**	150	380	320	550	543	875	249	213
Alien†	1850	450	950	750	1600	1131	**2071**	250	260
Asteroids†	**2000**	550	850	850	1100	1473	1444	750	730
Atlantis†	**2.3M**	150k	450k	350k	650k	707.8k	980.2k	15.0k	15.2k
IceHockey†	**-1.9**	-15.0	-12.0	-11.0	-8.5	-7.7	-3.4	-10.0	-9.8
Median HNS (11 seen)	0.74	0.06	0.36	0.17	0.85	0.31	1.07	0.05	0.04
Mean HNS (11 seen)	2.51	0.14	0.53	0.30	0.91	0.54	2.02	0.13	0.07
Median HNS (4 held-out)	0.50	0.01	0.05	0.05	0.21	0.21	0.45	0.05	0.06

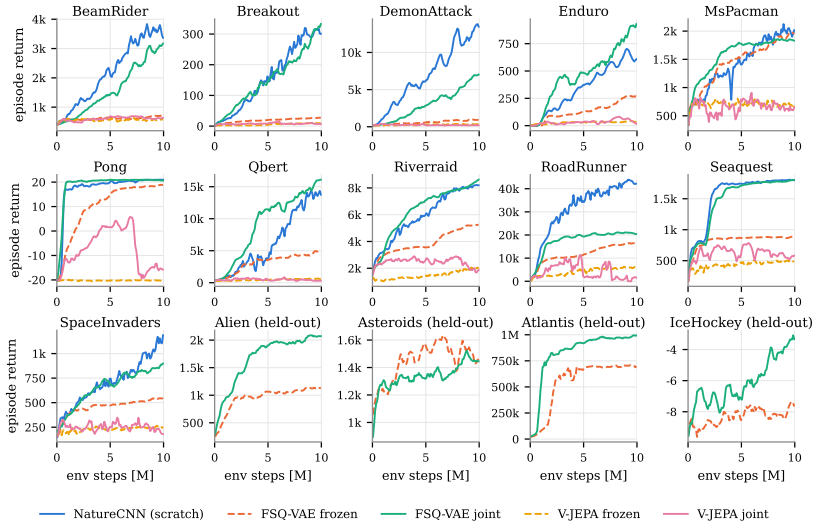


Fig. 4: Learning curves over 10 M agent steps (100 bins; FSQ curves smoothed over five). Solid: trained or fine-tuned encoders; dashed: frozen encoders.

Table 5: Sample efficiency: million steps until the learning curve first covers 50% and 80% of the way from NatureCNN's initial to its final return ("–": never). FSQ-J/F: FSQ-VAE joint/frozen. Fastest in bold.

	50% level			80% level		
Game	Nature	FSQ-J	FSQ-F	Nature	FSQ-J	FSQ-F
BeamRider	**4.3**	6.8	–	**6.8**	8.6	–
Breakout	**5.0**	5.2	–	**7.0**	8.2	–
DemonAttack	**5.2**	9.3	–	**7.5**	–	–
Enduro	4.3	**2.2**	–	7.2	**5.8**	–
MsPacman	2.5	**1.9**	2.6	6.2	**3.8**	7.2
Pong	0.7	**0.6**	1.8	**0.7**	0.8	4.2
Qbert	6.0	**3.5**	–	7.3	**4.7**	–
Riverraid	2.9	**2.5**	8.8	6.2	**4.8**	–
RoadRunner	**1.9**	–	–	**4.0**	–	–
Seaquest	**1.9**	2.0	–	**2.2**	3.1	–
SpaceInvaders	**3.6**	4.0	–	**8.0**	–	–
FSQ joint faster		5/11			4/11	

V-JEPA. Both V-JEPA agents learned very little (median HNS 0.05 frozen and 0.04 fine-tuned), even though the frozen critic predicted the returns reasonably well (explained variance 0.56–0.97). Fine-tuning was also unstable: on Pong, the agent reached +8.4 after 7 M steps and then dropped back to −15. Suspecting that the 9 216-dimensional feature was too large, I repeated all 22 runs with the tokens averaged into a 256-dimensional feature and with separate encoders for actor and critic, and I repeated the frozen runs once more with a five times larger entropy bonus (Table 6). Neither change helped: the pooled frozen agent beat the flattened one on only one game, the pooled fine-tuned policy became almost deterministic on ten games, and the larger entropy bonus kept the policy random but prevented the critic from learning.

Table 6: V-JEPA feature ablation (11 seen games): median HNS of the training return and of the final policy on 30 test episodes, end-of-training critic explained variance and policy entropy, and games on which the pooled variant beats the flattened one.

			median HNS		expl.			
Feature	mode	ent. coef.	train	test	var.	entropy	FPS	wins
9 216 (flat)	frozen	0.01	0.045	0.043	0.73	1.53	854	–
9 216 (flat)	joint	0.01	0.043	0.028	0.78	1.12	677	–
256 (pooled)	frozen	0.01	0.021	0.034	0.53	1.54	825	1/11
256 (pooled)	joint	0.01	0.007	0.000	-0.27	0.18	640	3/11
256 (pooled)	frozen	0.05	0.015	0.014	0.03	2.12	859	0/11

Frozen vs. Fine-Tuned and Unseen Games. For the FSQ encoder, fine-tuning outperformed freezing on 13 of 15 games (all except MsPacman and Asteroids), in some cases by a large factor (Breakout 12×, DemonAttack 7×). On the held-out games, it was better on three of four (median HNS 0.45 vs. 0.21). However, the NatureCNN trained from scratch on these games was ahead of it on Asteroids, Atlantis and IceHockey (median HNS 0.50), and the pre-trained encoder was better only on Alien (2 071 vs. 1 850). All other pre-trained encoders scored lower still (median HNS 0.01–0.21). Pre-training on eleven games therefore gave no advantage on unseen games, although it did not prevent the agent from learning them.

7.3 World-Model Diagnostics

A high similarity between predicted and actual latents can be misleading. A collapsed encoder makes all latents similar, and since s_{t+1} shares three frames with s_t, copying z_t already achieves a high score. I therefore evaluated the V-JEPA on 64 sequences of five steps and compared it against a copy baseline, an untrained model, and deliberately wrong actions (Table 7).

Table 7: V-JEPA diagnostics: seen games (validation split, true game embedding) and held-out games (UNKNOWN embedding); chance or untrained values in brackets.

Diagnostic	11 seen games	4 held-out games
Cos. sim. predictor / identity, $k = 1$	0.987 / 0.974	0.939 / 0.971
Cos. sim. predictor / identity, $k = 5$	0.961 / 0.915	0.806 / 0.912
Action sensitivity: sim. true − wrong action	+0.0057	+0.0001
Attention on action / game token (uniform 0.026)	0.029 / 0.046	0.026 / 0.055
Inverse-dynamics accuracy	55.9% (12.9%)	1.8% (5.6%)
Effective rank / 256	52.5 (2.5)	29.9 (1.5)

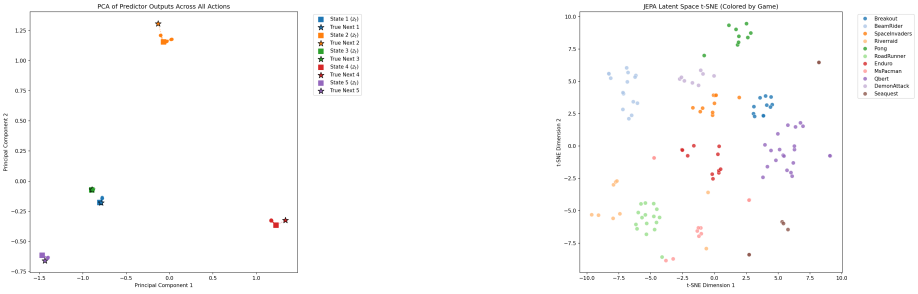
On the seen games, the latent space did not collapse (effective rank 52.5 compared with 2.5 for an untrained model), and the predictor outperformed copying at every step. The gap increased from +0.013 after one step to +0.046 after five steps, when input and target no longer share any frames, so the model learned genuine dynamics. The action, however, had very little influence: giving the predictor a wrong action changed its output by only 0.006, and the action token received barely more attention than an average token. The game token received more attention (0.046, almost twice uniform), but the encoder could already identify the game from the pixels: the latents form clusters by game (Figure 5b). On the held-out games, the predictor was worse than copying and the inverse head was below chance, so the model learned the eleven training games rather than Atari in general.

7.4 Why the V-JEPA Failed with PPO

Measured by its own training objectives, the V-JEPA performed well, yet PPO could not make use of its features. Table 8 summarises the collected evidence, and Figure 5a illustrates the central problem.

Table 8: Evidence for why the V-JEPA encoder did not support PPO (from Sections 7.1 and 7.3, Table 6 and Figure 5b).

Explanation	Evidence	Consequence for PPO
Dynamics ignore the action	true vs. wrong action: Δcos. 0.006; action-token attention 0.029 (uniform 0.026); Figure 5a	latent need not encode what the agent controls
Latents change slowly	$\cos(z_t, \bar{z}_{t+1}) = 0.974$; 0.915 five steps later; 14×14 patches	small, decisive sprite motion barely moves the feature
Organised by game	latents cluster by game (Figure 5b); game token gets 1.8× uniform attention	capacity spent on information that is constant within a run
Specialised to pre-training games	held-out loss 3× seen; held-out inverse accuracy below chance	tuned to expert states of eleven games
State, not action, information	critic expl. var. 0.56–0.97 while policy entropy stays high (1.53)	value readable, best action not
Little action variety in the data	expert's top action: 45% (Qbert), 67% (RoadRunner) of steps (Table 1)	few examples of what *other* actions would change
Not size or exploration	pooled 256-d feature, entropy bonus 0.05: no gain	rules out the simplest remedies



(a) PCA, predictions for all actions

(b) t-SNE, coloured by game

Fig. 5: V-JEPA latents. (a) PCA for five states: context z_t (square), true next latent (star) and the prediction for every valid action (dots). The predictions for all actions form one tight cluster near z_t: the predictor hardly distinguishes between actions. (b) t-SNE of latents from the eleven seen games: they group by game.

In my view, the main cause is a mismatch between what the JEPA is trained to do and what a policy needs. A JEPA predicts what is *predictable* about the next frame and is free to ignore details that have little effect on the latent error [5,6]. In Atari, however, such details, like the exact position of the ball or a bullet, are what the agent needs to act. A representation suited for control should capture what the agent's actions change [21], which is what my predictor modelled worst (Figure 5a). The inverse loss was meant to counter this, but only trains the encoder (Table 2), and the expert data contains few examples of

alternative actions. The overlapping frames and the slowly moving EMA target push the model towards latents that change very little from one step to the next. Such features suffice to estimate the value of a state (the critic's explained variance was high), but are too smooth to separate states whose best actions differ by a few pixels. Finally, fine-tuning a 12.5 M-parameter transformer with PPO's learning rate was unstable, which is consistent with reports that pre-trained visual features transfer poorly to control [15,14].

8 Conclusion

In this project, I investigated whether a single pre-trained encoder can serve as a universal feature extractor for Atari games. I pre-trained VAE, VQ-VAE, FSQ-VAE and V-JEPA encoders on expert data from eleven games and used them with PPO for 10 M steps on these and four unseen games, frozen or fine-tuned. The results give a partly positive answer.

The FSQ-VAE, pre-trained on eleven games without rewards, outperformed the Standard VAE on every game and the Residual VQ-VAE on all but one, and proved to be a good initialisation (**RQ1**). Fine-tuned with PPO, it achieved a higher median score than a NatureCNN trained from scratch and learned faster on four games, although its final score was higher on only four of eleven games and its mean score was lower. Frozen, it reached less than a third of that median, so fine-tuning was clearly preferable (**RQ2**): a frozen encoder would need to keep every small detail that matters in every game, and a reward-free objective gives it no reason to do so. No pre-trained encoder gave an advantage on the unseen games, where the NatureCNN trained from scratch was ahead of the fine-tuned FSQ-VAE on three of four (**RQ3**); this is not surprising given only eleven pre-training games and quite different held-out games (Asteroids is 95% black). Freezing the V-JEPA encoder made PPO about a quarter faster (854 vs. 677 steps per second), but it remained slower than the NatureCNN (976), so freezing did not make the agent cheaper than training from scratch. The FSQ-VAE is easier to interpret, because its decoder shows what the features contain (**RQ4**).

The V-JEPA learned a non-collapsed latent space and meaningful dynamics, but its objective rewarded predicting what usually happens in a game rather than what the agent's actions change, so its features were not useful for PPO, whether flattened or pooled. A practical lesson was that every bug in the V-JEPA pipeline still produced a plausible loss curve, so checks such as the copy baseline and wrong actions were necessary.

The main limitations are the single seed and PPO hyper-parameters that were not re-tuned for the pre-trained encoders. As a next step, I would repeat the main comparison with three seeds. For the V-JEPA, smaller patches and a training signal in which the action has a real effect (e.g. multi-step prediction or a reward head, as in TD-MPC2) appear most promising, and the features should first be validated by predicting game-state variables such as object positions. Using the FSQ grid inside a world model [10,11] and pre-training on more games

are further directions. A frozen, truly universal Atari feature extractor remains an open problem.

Acknowledgements. I would like to thank Oleh Shkalikov for proposing and supervising this project, and the Centre for Information Services and High Performance Computing (ZIH) at TU Dresden for the GPU time on the *Capella* cluster.

References

1. Machado, M.C., Bellemare, M.G., Talvitie, E., Veness, J., Hausknecht, M., Bowling, M.: Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. Journal of Artificial Intelligence Research **61**, 523–562 (2018)
2. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., et al.: Human-level control through deep reinforcement learning. Nature **518**(7540), 529–533 (2015)
3. van den Oord, A., Vinyals, O., Kavukcuoglu, K.: Neural discrete representation learning. In: Advances in Neural Information Processing Systems (NeurIPS) (2017)
4. Mentzer, F., Minnen, D., Agustsson, E., Tschannen, M.: Finite scalar quantization: VQ-VAE made simple. In: International Conference on Learning Representations (ICLR) (2024)
5. LeCun, Y.: A path towards autonomous machine intelligence. OpenReview preprint (version 0.9.2) (2022)
6. Assran, M., Duval, Q., Misra, I., Bojanowski, P., Vincent, P., Rabbat, M., LeCun, Y., Ballas, N.: Self-supervised learning from images with a joint-embedding predictive architecture. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 15619–15629 (2023)
7. Bardes, A., Garrido, Q., Ponce, J., Chen, X., Rabbat, M., LeCun, Y., Assran, M., Ballas, N.: Revisiting feature prediction for learning visual representations from video. Transactions on Machine Learning Research (2024)
8. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347 (2017)
9. Raffin, A.: RL baselines3 zoo. https://github.com/DLR-RM/rl-baselines3-zoo (2020)
10. Hafner, D., Lillicrap, T., Norouzi, M., Ba, J.: Mastering Atari with discrete world models. In: International Conference on Learning Representations (ICLR) (2021)
11. Micheli, V., Alonso, E., Fleuret, F.: Transformers are sample-efficient world models. In: International Conference on Learning Representations (ICLR) (2023)
12. Stooke, A., Lee, K., Abbeel, P., Laskin, M.: Decoupling representation learning from reinforcement learning. In: International Conference on Machine Learning (ICML). pp. 9870–9879 (2021)
13. Schwarzer, M., Anand, A., Goel, R., Hjelm, R.D., Courville, A., Bachman, P.: Data-efficient reinforcement learning with self-predictive representations. In: International Conference on Learning Representations (ICLR) (2021)
14. Parisi, S., Rajeswaran, A., Purushwalkam, S., Gupta, A.: The unsurprising effectiveness of pre-trained vision models for control. In: International Conference on Machine Learning (ICML). pp. 17359–17371 (2022)

15. Hansen, N., Yuan, Z., Ze, Y., Mu, T., Rajeswaran, A., Su, H., Xu, H., Wang, X.: On pre-training for visuo-motor control: Revisiting a learning-from-scratch baseline. In: International Conference on Machine Learning (ICML) (2023)
16. Hansen, N., Su, H., Wang, X.: TD-MPC2: Scalable, robust world models for continuous control. In: International Conference on Learning Representations (ICLR) (2024)
17. Kingma, D.P., Welling, M.: Auto-encoding variational Bayes. In: International Conference on Learning Representations (ICLR) (2014)
18. Pathak, D., Agrawal, P., Efros, A.A., Darrell, T.: Curiosity-driven exploration by self-supervised prediction. In: International Conference on Machine Learning (ICML). pp. 2778–2787 (2017)
19. Badia, A.P., Piot, B., Kapturowski, S., Sprechmann, P., Vitvitskyi, A., Guo, Z.D., Blundell, C.: Agent57: Outperforming the Atari human benchmark. In: International Conference on Machine Learning (ICML). pp. 507–517 (2020)
20. Agarwal, R., Schwarzer, M., Castro, P.S., Courville, A., Bellemare, M.G.: Deep reinforcement learning at the edge of the statistical precipice. In: Advances in Neural Information Processing Systems (NeurIPS) (2021)
21. Gelada, C., Kumar, S., Buckman, J., Nachum, O., Bellemare, M.G.: DeepMDP: Learning continuous latent space models for representation learning. In: International Conference on Machine Learning (ICML). pp. 2170–2179 (2019)